# Situación

Si realizo la siguiente consulta en una tabla con 10 millones de filas.

```
SELECT * FROM usuarios
WHERE email = azul@gmail.com'
```

¿Qué se imaginan que hace la base de datos paso a paso por dentro para darles la respuesta en milisegundos?


FICTION: L-R
LATIN AMERICAN LIT.
LATIN AMERICAN LIT.
LATIN AMERICAN LIT.

Clase : Índices

**Temas:**

- Índices Ordenados
  - Índices Ordenados Densos
  - Índices Ordenados Dispersos
  - Índices Multinivel
  - Índices Secundarios
- Índices Asociativos
- Técnica de Indexación
- Ejercicio

# ¿Qué es un índice?

Los índices de los sistemas de bases de datos juegan el mismo papel que los índices de los libros o los catálogos de fichas de las bibliotecas.

El sistema de bases de datos busca en un **índice para encontrar el bloque de disco en que se encuentra el registro correspondiente que necesita**.

# ¿Qué tipos hay?

Índices ordenados/primarios (B-Trees)

Están ordenados secuencialmente según alguna clave de búsqueda.

Estos archivos con índice primario según una clave de búsqueda se llaman archivos secuenciales indexados.

Índices asociativos (hash indices)

Estos índices están basados en una distribución uniforme de los valores a través de una serie de cajones (buckets).

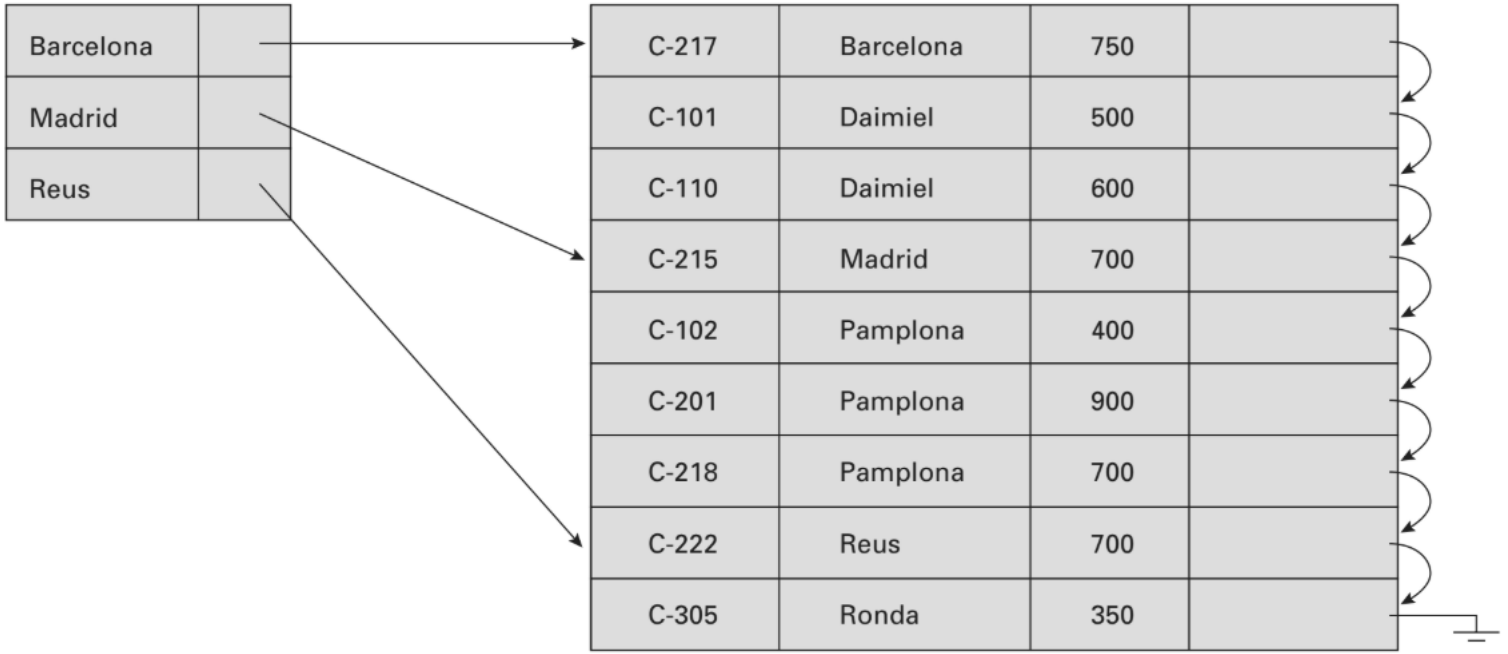
El valor asignado a cada cajón está determinado por una función, llamada función de asociación (hash function).

Los atributos o conjunto de atributos usados para buscar en un archivo se llaman claves de búsqueda.

# Índices Ordenados/Primarios

En las tablas, valores están almacenados secuencialmente según la clave de búsqueda.

Se generan archivos con índice primario según la clave de búsqueda se llaman **archivos secuenciales indexados**.




Los registros están almacenados según el orden de la clave de búsqueda, siendo esta clave **nombre-sucursal**.

# Índices Ordenados/Primarios

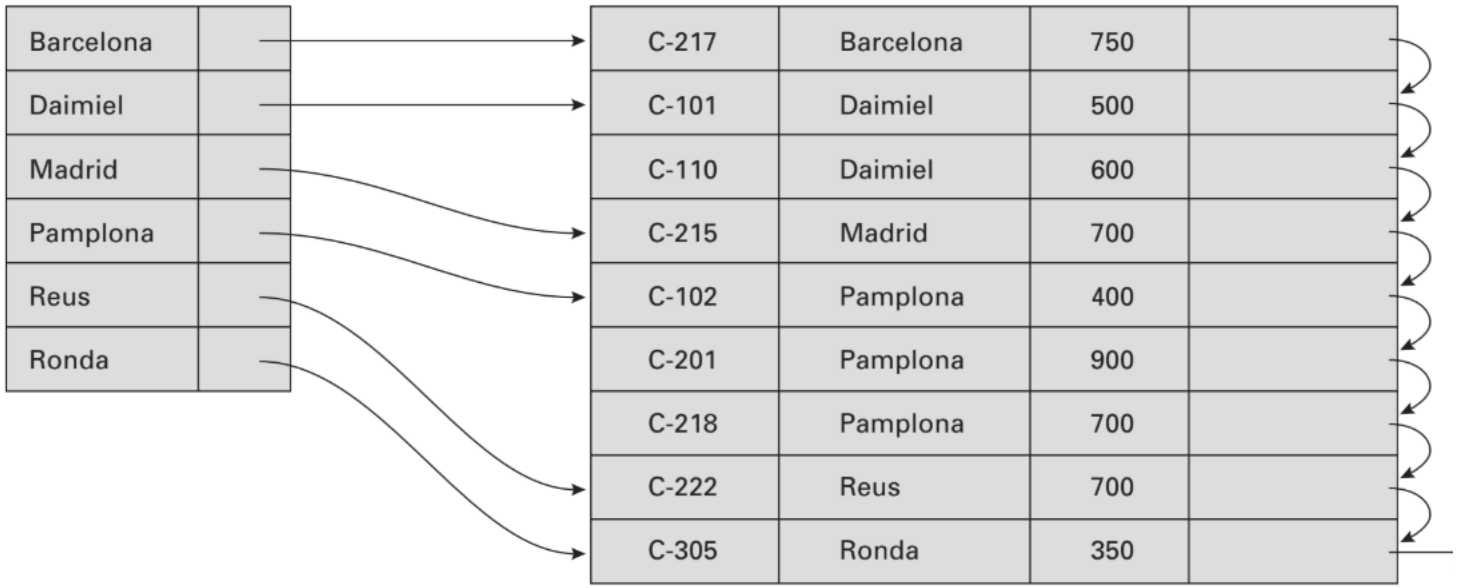
En los archivos de índice contienen:

- Registro índice o entrada del índice , que consiste en un valor de la clave de búsqueda
- Puntero a un registro, con ese valor de la clave de búsqueda. consiste en el identificador de un bloque de disco y un desplazamiento en el bloque de disco para identificar el registro dentro del bloque.

Hay dos clases de índices ordenados que se pueden emplear:

• **Índice denso**.
Aparece un registro índice por cada valor de la clave de búsqueda en el archivo. El registro índice contiene el valor de la clave y un puntero **al primer registro con ese valor de la clave de búsqueda**.

El resto de registros con el mismo valor de la clave de búsqueda se almacenan consecutivamente después del primer registro, dado que, ya que el índice es primario, los registros se ordenan sobre la misma clave de búsqueda.



# Índices Ordenados/Primarios

En los archivos de índice contienen:

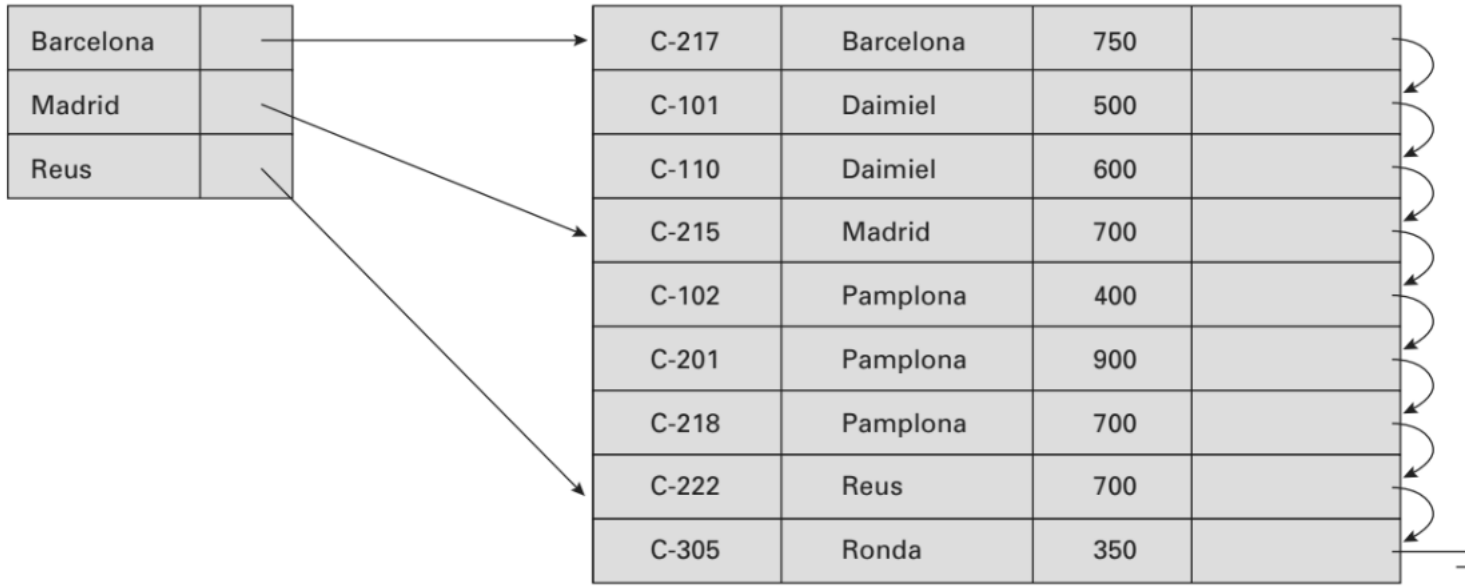
- Registro índice o entrada del índice , que consiste en un valor de la clave de búsqueda
- Puntero a un registro, con ese valor de la clave de búsqueda. consiste en el identificador de un bloque de disco y un desplazamiento en el bloque de disco para identificar el registro dentro del bloque.

Hay dos clases de índices ordenados que se pueden emplear:

- **Índice disperso**

Sólo se crea un registro índice para algunos de los valores.
Al igual que en los índices densos, cada registro índice contiene un valor de la clave de búsqueda y un puntero al primer registro con ese valor de la clave.

Para localizar un registro se **busca la entrada del índice con el valor más grande que sea menor o igual que el valor que se está buscando**. Se empieza por el registro apuntado por esa entrada del índice y se continúa con los punteros del archivo hasta encontrar el registro deseado.



# Índices Ordenados/Primarios

Cada vez que haces un INSERT, UPDATE o DELETE, el motor actualiza de forma transparente e inmediata el índice primario para mantener el orden.

Generalmente es **más rápido localizar un registro si se usa un índice denso en vez de un índice disperso**. Sin embargo, **los índices dispersos** tienen algunas ventajas sobre los índices densos, como el **utilizar un espacio más reducido y un mantenimiento adicional menor para las inserciones y borrados**.

# Situación

Si realizo la siguiente consulta en una tabla con 10 millones de filas.

```
SELECT * FROM usuarios
WHERE id = 1324
```

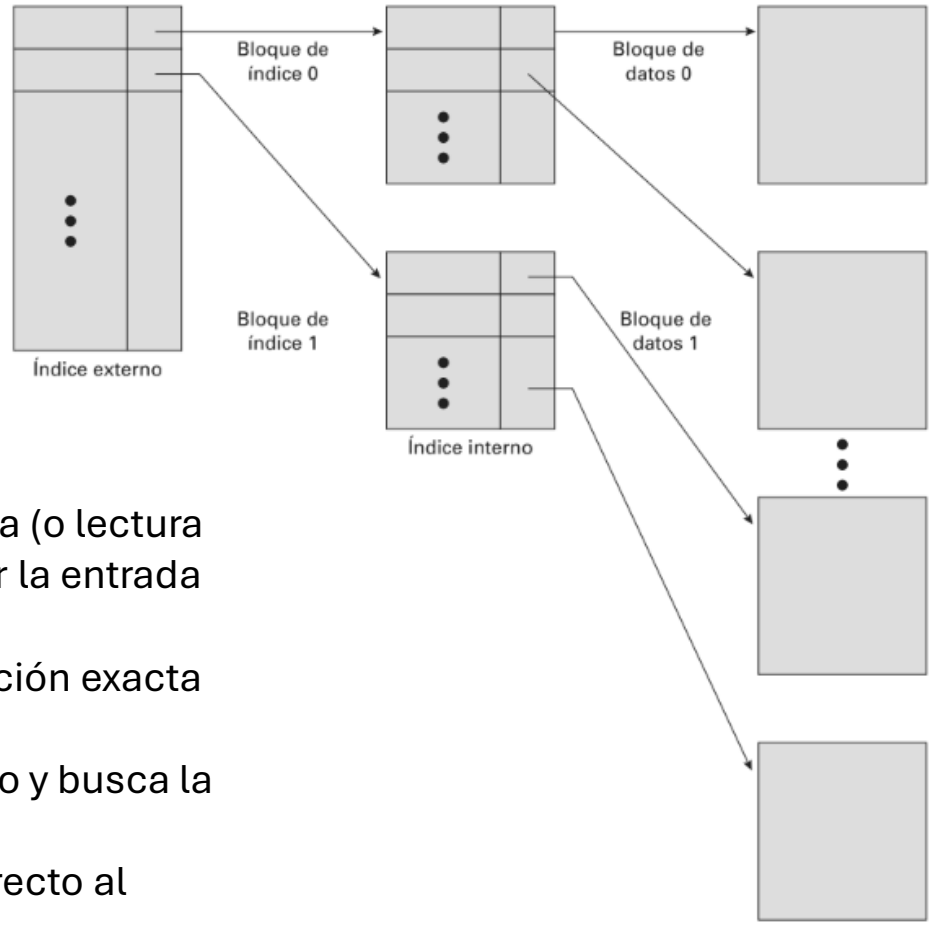
¿Cuál de los dos tipos de índices que vimos es mas conveniente?

# Índices Multinivel

Los índices multinivel sirven para construir un índice sobre el propio índice.

**Índice Interno:** Es el índice primario original construido sobre el archivo de datos real.

**Índice Externo:** Es un **índice disperso** construido sobre el *Índice Intern*




**¿Cómo lo hace?**

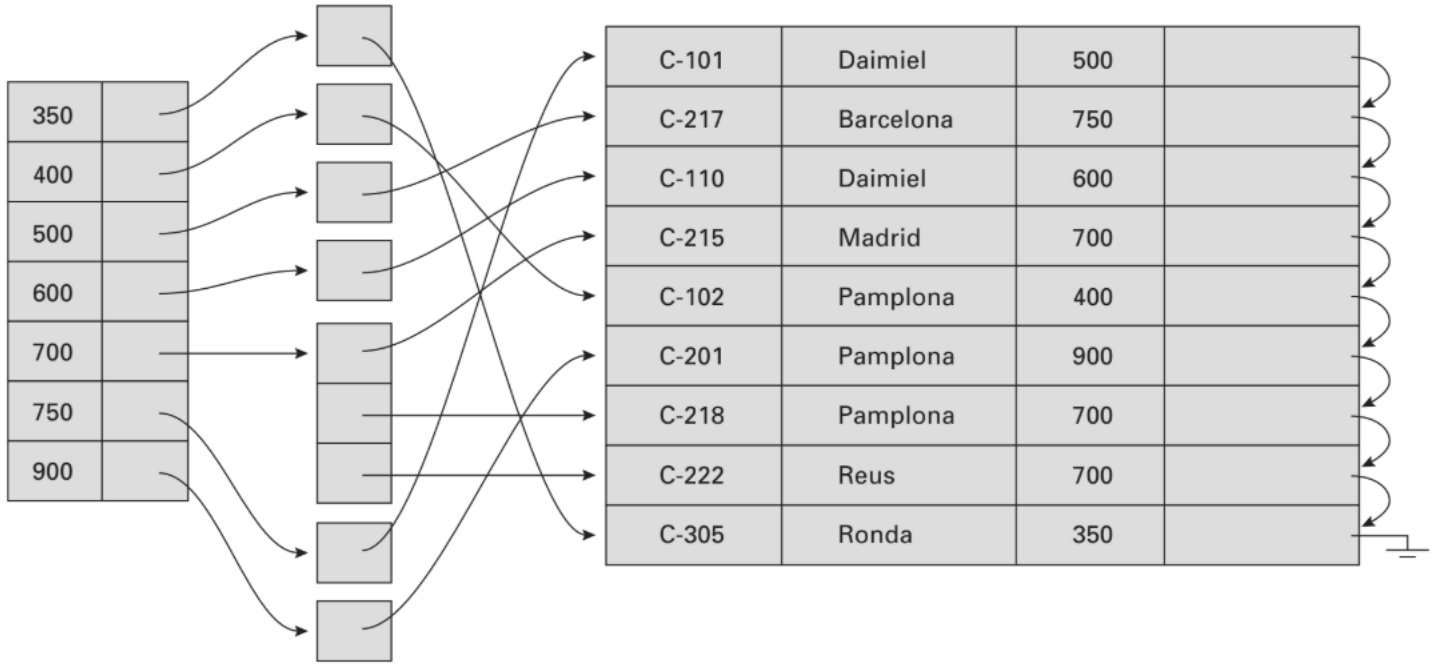
- **Consulta el Índice Externo:** Realiza una búsqueda binaria (o lectura directa si está en RAM) en el *índice externo* para encontrar la entrada correspondiente al bloque del *índice interno*.
- **Obtiene el Puntero:** El índice externo le devuelve la dirección exacta del bloque de disco del *índice interno*.
- **Lee el Índice Interno:** Accede a ese único bloque en disco y busca la entrada del registro deseado.
- **Accede al Dato Real:** El índice interno le da el puntero directo al bloque de datos final.

# Índices Secundarios

Mientras que solo puede existir **un** índice primario por tabla (porque los datos solo se pueden ordenar físicamente de una sola forma), una tabla puede tener **múltiples índices secundarios**.

¿Para qué sirven?

No siempre cuando buscamos un registro lo hacemos por su primary index, sino que **muchas veces buscamos registros mediante otros campos**, es por eso que se crea un índice secundario.




¿Cómo funcionan?

El índice secundario apunta al **Índice Primario**, y el Índice Primario es el que finalmente te entrega los datos reales

# Ejemplo: Indice Primario

Si realizo la siguiente consulta en una tabla con 10 millones de filas.

```
SELECT * FROM customer
WHERE customer_id = 14
```

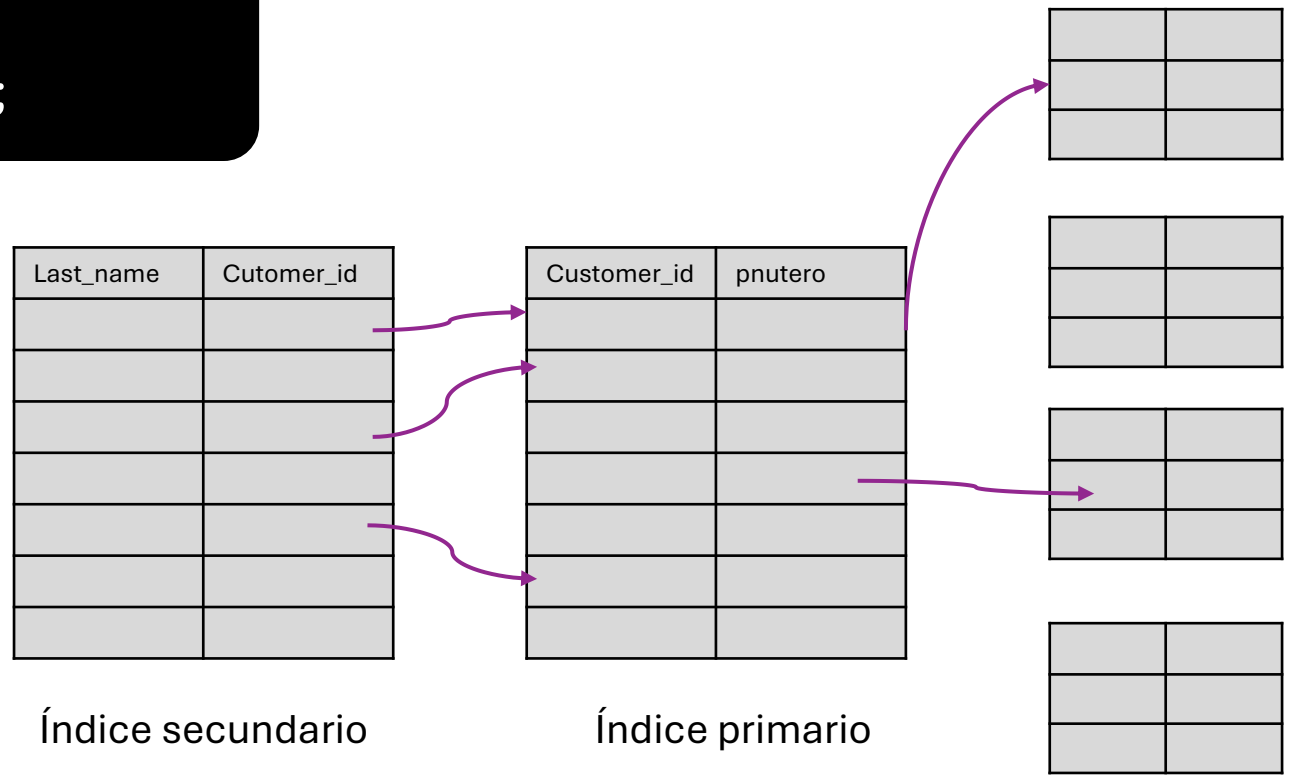
- La base de datos va **directamente al Índice Primario**.
- Encuentra la clave 14  y ahí mismo tiene **toda la fila con todos los datos** (nombre, email, dirección).
- **Costo:** 1 sola búsqueda en el árbol.

# Ejemplo: Indice Secundario

Si realizo la siguiente consulta

```
SELECT * FROM customer
WHERE last_name = 'GUINNESS';
```

- Consulta en el ÍNDICE SECUNDARIO (last_name) [ 'GUINNESS' ] y Devuelve la Clave Primaria: (customer_id = 1)
- Consulta en el ÍNDICE PRIMARIO (customer_id) [ customer_id = 1 ] Obtiene la FILA COMPLETA con todos los datos



# Situacion

¿Si busco en la tabla por un campo que no está contemplado como un índice primario o secundario?

El motor debe ejecutar la operación más pesada en rendimiento: el **Full Table Scan** (Escaneo Completo de Tabla)

**El motor recorre todo el archivo:** Debe leer **absolutamente todos los bloques de disco de la tabla** hasta llegar a la última fila, porque no tiene forma de saber de antemano cuántos registos bajo una condición existen ni dónde están guardados

# Índices en Tablas

```
1  CREATE TABLE `customer` (
2      `customer_id` smallint unsigned NOT NULL AUTO_INCREME
3      `store_id` tinyint unsigned NOT NULL,
4      `first_name` varchar(45) NOT NULL,
5      `last_name` varchar(45) NOT NULL,
6      `email` varchar(50) DEFAULT NULL,
7      `address_id` smallint unsigned NOT NULL,
8      `active` tinyint(1) NOT NULL DEFAULT '1',
9      `create_date` datetime NOT NULL,
10     `last_update` timestamp NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TI
11     PRIMARY KEY (`customer_id`),
12     KEY `idx_fk_store_id` (`store_id`),
13     KEY `idx_fk_address_id` (`address_id`),
14     KEY `idx_last_name` (`last_name`),
15     CONSTRAINT `fk_customer_address` FOREIGN KE
16     CONSTRAINT `fk_customer_store` FOREIGN KEY (`store_id`) REFERENCES `store`
17 ) ENGINE=InnoDB AUTO_INCREMENT=600 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0
```

```
SHOW INDEX FROM customer;
```

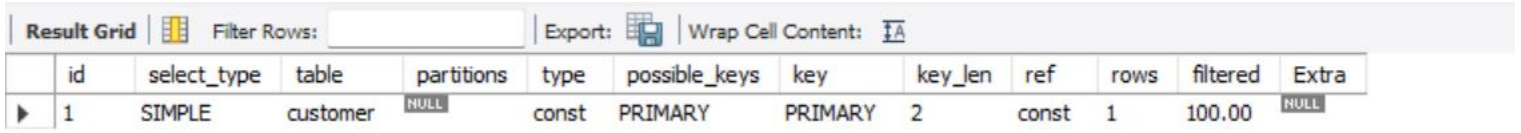
```
SHOW CREATE TABLE customer;
```

- **Indice primario**
- **Índices secundarios**

SAKILA EJEMPLOS

# Búsqueda por índice primario

```
EXPLAIN SELECT * FROM customer
WHERE customer_id = 5;
```



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

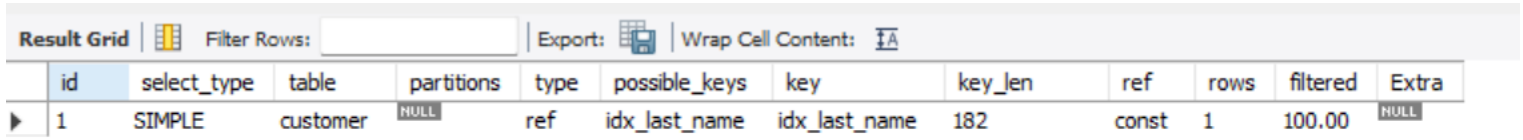
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | SIMPLE | customer | NULL | const | PRIMARY | PRIMARY | 2 | const | 1 | 100.00 | NULL |

- **type = const:** Significa que la consulta hizo búsqueda por índice primario
- **key = PRIMARY:** MySQL usó directamente el **Índice Primario**.
- **rows = 1:** El motor fue tan preciso que solo tuvo que examinar **1 sola fila**.

SAKILA EJEMPLOS

# Búsqueda por índice secundario

```
EXPLAIN SELECT * FROM customer
WHERE last_name = 'MONROE';
```



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

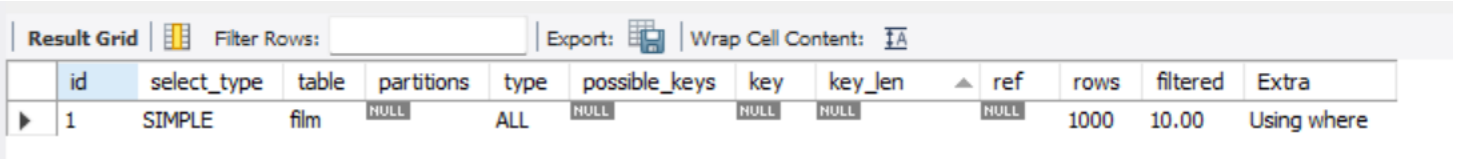
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | SIMPLE | customer | NULL | ref | idx_last_name | idx_last_name | 182 | const | 1 | 100.00 | NULL |

- **type = ref:** Indica que se usó un índice secundario para encontrar las coincidencias.
- **key = idx_last_name:** Usó el **Índice Secundario**.

SAKILA EJEMPLOS

# Búsqueda por índice secundario

```
EXPLAIN SELECT * FROM film
WHERE length = 120;
```



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | SIMPLE | film | NULL | ALL | NULL | NULL | NULL | NULL | 1000 | 10.00 | Using where |

- **type = ALL:** significa *Full Table Scan*.
- **key = NULL:** No existe ningún índice disponible para ayudar a la consulta.
- **rows = 1000:** La tabla film tiene 1.000 filas y MySQL **tuvo que leer las 1.000 filas una por una** en disco para ver cuáles duraban 120 minutos.

SAKILA EJEMPLOS

# Creación de índices

```
CREATE INDEX nombre_del_indice ON nombre_tabla (columna);
```

# Creación de índices

```
ALTER TABLE nombre_tabla ADD INDEX nombre_del_indice (columna);
```

¿Recuerdan la consulta anterior, donde buscar películas por su duración (`length`) provocaba un **Full Table Scan** (`type = ALL`) porque la columna no tenía índice?

Se deberá crear en Sakila

# Índices

¿Qué vimos la clase pasada?

# Índices Asociativos

En un **índice asociativo (o hashing)**, no hay comparaciones ni orden. Se **asocia** directamente la clave de búsqueda con una ubicación física en el disco a través de una fórmula matemática.

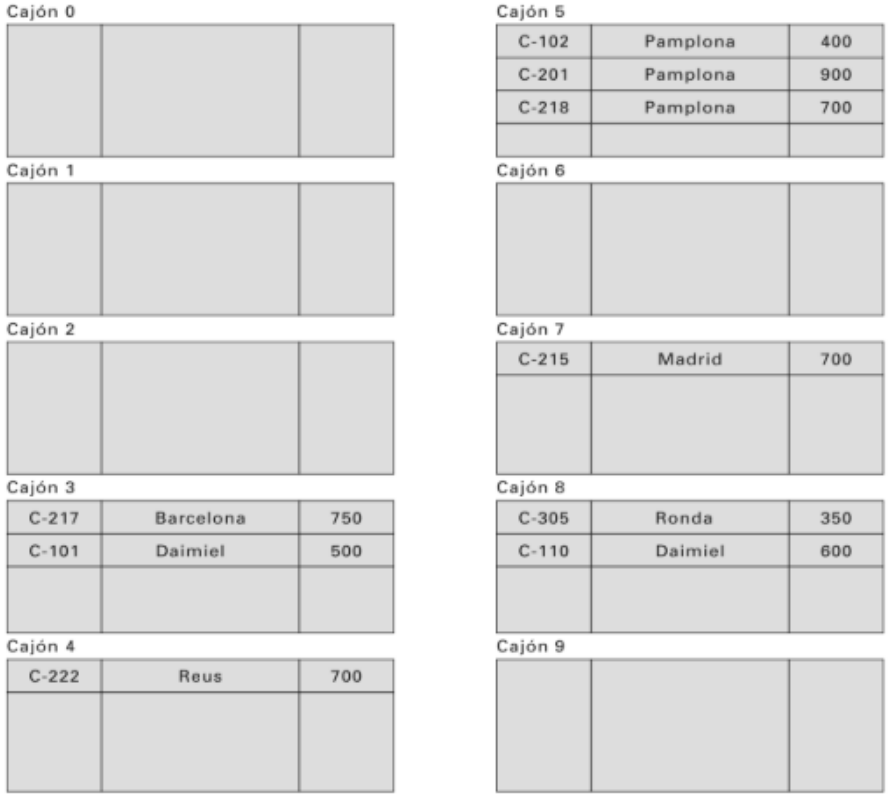
**La fórmula clave:**

$$\text{Dirección del bucket} = h(K)$$

Donde $K$ es el valor que buscas (ej. un DNI o ID) y $h$ es la **Función Hash**.

# Índices Asociativos - Principios

- Estos índices están **basados en una distribución uniforme de los valores a través de una serie de cajones (buckets**).
- El valor asignado a cada cajón está determinado por una función, llamada **función de asociación (hash function).**
- En una organización de archivos por asociación se obtiene la **dirección del bloque de disco que contiene el registro deseado mediante el cálculo** directo de una función sobre el valor de la clave búsqueda del registro.
- En nuestra descripción de asociación, utilizaremos el término cajón (**bucket**) para indicar **una unidad de almacenamiento que puede guardar uno o más registros**.



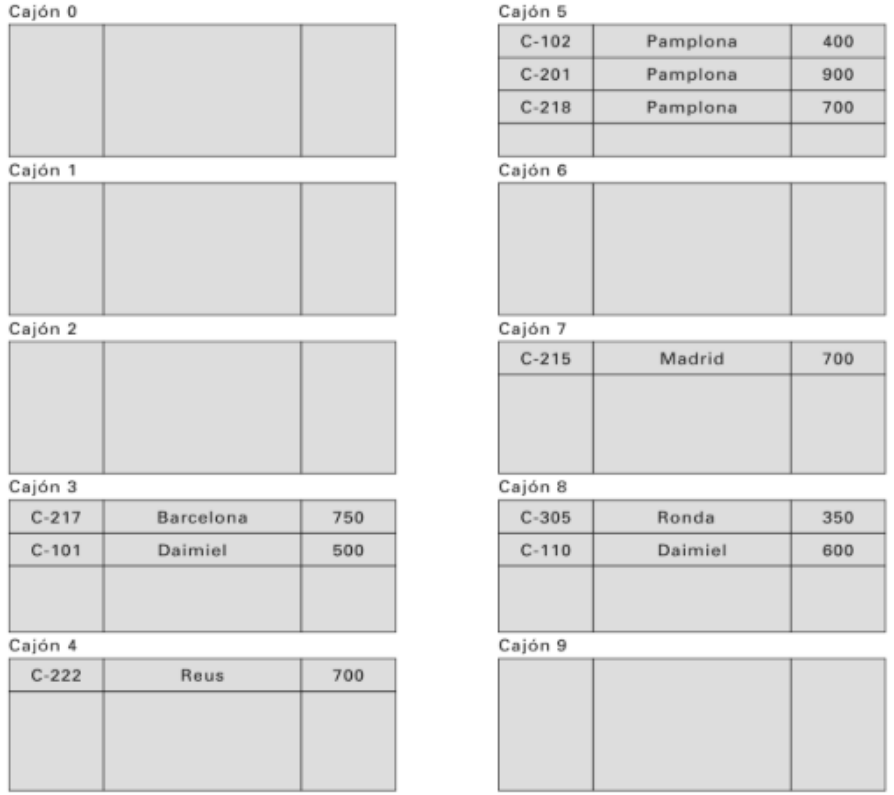
# Función de Asociación

Una función de asociación ideal distribuye las claves almacenadas uniformemente a través de los cajones para que cada uno de ellos tenga el mismo número de registros.

Se pretende elegir una función de asociación que asigne los valores de las claves de búsqueda a los

Cajones de manera que se cumpla lo siguiente:

- Distribucion Uniforme : Cada balde (bucket) tiene asignada exactamente la misma cantidad de claves de búsqueda posibles dentro del conjunto total de valores que podrían existir.
- Distribucion aleatoria: cada balde recibe aproximadamente la misma cantidad de registros, **independientemente de cómo estén distribuidas las claves en el archivo real**.



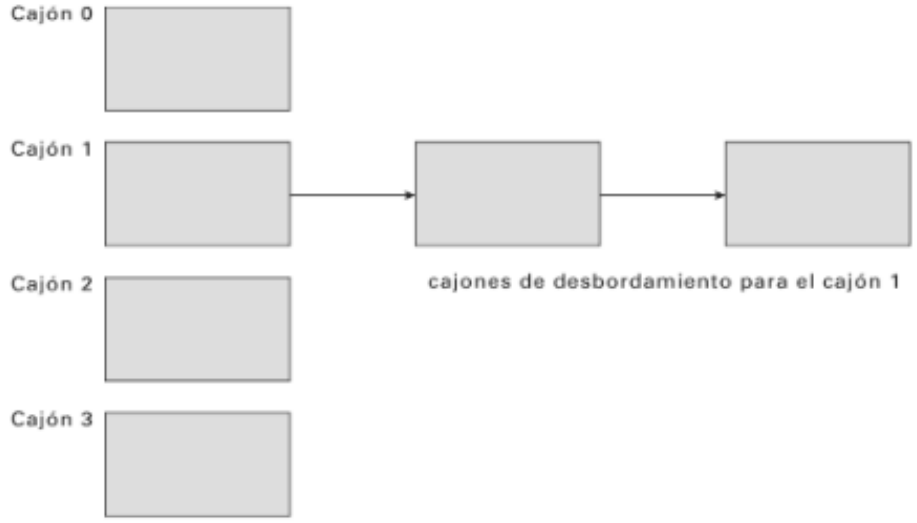
# Función de Asociación – Desbordamiento de buckets

**¿Qué hacemos cuando un *bucket* se llena y tenemos que meter un registro más ahí?**

A esto se le conoce como **desbordamiento de buckets (*bucket overflow*)**. Si no se gestiona, la base de datos simplemente colapsaría o perdería información.

**Causa**

- **Insuficiencia de cajones:** Al diseñar la base de datos se asignaron pocos cajones en total ($n_B$), por lo que, matemáticamente, los registros superan la capacidad física disponible.
- **Sesgo en la distribución (*Skew*):** Aunque haya cajones de sobra en total, algunos cajones se llenan mientras otros quedan vacíos. Esto pasa por:
  - **Claves no uniformes:** Hay muchos registros reales con la misma clave de búsqueda (ej. muchos clientes de la misma ciudad).
  - **Infección de la función hash:** La función hash no logra dispersar aleatoriamente los datos de forma justa.



# Función de Asociación – Desbordamiento de buckets

- Para resolver el desbordamiento sin mover los registros que ya estaban guardados, Silberschatz explica que se utiliza una estructura de **lista enlazada de cajones**.
- Cada **cajón principal** guarda sus datos y, además, reserva un espacio al final para guardar un **puntero** (una dirección de memoria/disco).
- Cuando el cajón principal se llena y llega un nuevo registro, la base de datos asigna un **cajón de desbordamiento** (un cajón auxiliar).
- El cajón principal usa su puntero para apelar a este nuevo cajón auxiliar.
- Si el cajón de desbordamiento también se llena, se engancha otro cajón de desbordamiento más al final.
- A toda esta secuencia de cajones enlazados entre sí se la denomina **cadena de desbordamiento**.

**El peligro de las cadenas largas:** Si un cajón tiene una **cadena de desbordamiento** con 3 cajones auxiliares, el motor podría necesitar hasta **4 lecturas de disco** para encontrar un único registro.

# ¿Qué pasa en la vida real?

**No en todos los motores puedes elegir libremente el tipo de índice**. La posibilidad de elegir un índice asociativo (HASH) o cualquier otro tipo depende directamente de dos factores:

1. El **motor de base de datos** (MySQL, PostgreSQL, Oracle, etc.).
2. El **motor de almacenamiento interno** (*Storage Engine*) que use la tabla.

**¿Qué control tiene el usuario/DBA?**

Aunque no escribes el algoritmo matemático, **sí tomas decisiones estratégicas** sobre cómo y cuándo usar la lógica asociativa:

**A. Eliges el TIPO de índice (B-Tree o Hash)**

**B. Particionamiento de Tablas (*Hash Partitioning*)**

# Índices - Resumen

| **Criterio** | **Índices Ordenados (Densos/Dispersos/Multinivel)** | **Índices Asociativos (Hash)** |
|---|---|---|
| **Búsqueda Exacta (=)** | Rápida | Ultra rápida |
| **Búsqueda por Rango (>, BETWEEN)** | **Excelente** | **Mala / No sirve** |
| **Estructura Interna** | Secuencial / Árboles (B+) | Cajones (*buckets*) y Funciones Hash |
| **Riesgo Principal** | Desbalanceo de árboles | Cadena de desbordamiento de cajones |